Lead QA — Take-Home Assignment

Playwright + TypeScript automation challenge

ROLE	TIME ALLOWED	STACK	DELIVERABLE
Lead QA Automation	**2 days**	**Playwright + TypeScript**	**GitHub repo + README**

OVERVIEW

Thank you for interviewing with Simplenight. This exercise is the hands-on step in our Lead QA process. It mirrors the real work of the role: building a clean, maintainable automation framework against our **staging** environment. We weigh **how you structure and document the framework** as heavily as whether the test passes.

OBJECTIVE

Create a test-automation framework using **Playwright and TypeScript** to automate the booking process on the Simplenight website. Structure it so the booking process for **any category** could be automated — not only the one specified below.

SCOPE — THE AUTOMATED TEST SHOULD COVER

1. Go to the staging homepage **https://wl.stg.simplenight.com/**.
2. Select the **Hotels** category from the available navbar options.
3. Perform a search within the selected category, using: **Location** Miami · **Dates** August 1–3 · **Guests** 1 Adult + 1 Child.
4. Select **Map view** for the search results.
5. Filter using the left panel: **Price Range** 100 – 1000+ (the slider's maximum displays as **"1000+"**, an open-ended cap) · **Guest Score** "Very Good".
6. Zoom in on the map view and select only **1 hotel** option.
7. On the hotel card, check that **Price** and **Guest Score** are within the filtered parameters.

ENVIRONMENT & NOTES

- **Access:** the staging site is publicly accessible for search — **no account or login is required** to complete steps 1–7.
- **Guests:** if the guest selector asks for the child's age, choose any valid age (e.g. 8).
- **Async UI:** the site is a single-page app and search results load asynchronously — rely on Playwright's web-first waits, not fixed sleeps.
- **Scope end:** the flow ends at validating the hotel card (step 7) — **no checkout or payment** is required.

REQUIREMENTS

- Use the **Page Object Model (POM)** to separate pages and their element locators.
- Separate **test data** (inputs such as location, dates, guests) from the test scripts.
- Separate **execution parameters** (e.g. environment parameters like the test URL) so the tests can run across different environments.
- Follow **TypeScript best practices** and make proper use of Playwright's capabilities.

WHAT WE VALUE

- A clean, extensible framework a team could inherit and grow.
- Idiomatic Playwright — web-first assertions, resilient locators, no hard-coded sleeps.
- A clear **README** and tidy repository structure.
- If you use AI tools (Cursor, Claude, MCP servers, etc.) in your workflow, we welcome it — briefly tell us how you used them and how you kept quality under control.

SUBMISSION

- Provide a **GitHub repository** with the project (public, or shared with the panel).
- Include a **README.md** covering: how to install dependencies; how to configure and run the tests (with the exact commands); any additional setup required.
- **Reply to all** on the assignment email thread with your repository link.

Time allowed: a maximum of **2 days** from receipt.
Return by: _____ (EOD EST)
Questions: reply on the email thread and we'll get back to you promptly.